МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №4. Бинарные деревья

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151438 _____ Д.А, Пузынин
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

- изучить метод хеширования на примере организации таблиц идентификаторов в процессе трансляции программ;
- получить представление о преимуществах и недостатках, присущих различным методам организации таблиц символов (идентификаторов).

ПОСТАНОВКА ЗАДАЧИ

Общая постановка задачи:

Для выполнения лабораторной работы требуется написать программу, которая получает на входе набор идентификаторов, организует таблицу по заданному методу и позволяет осуществить многократный поиск идентификатора в этой таблице. Список идентификаторов считать заданным в виде текстового файла. Длина идентификаторов ограничена 32 символами.

Требования к программе:

- разработать и использовать шаблон класса «Хеш-таблица»;
- интерфейс программы представить в виде меню (загрузка/выгрузка файла, поиск, добавление, удаление, вывод на экран);
- при поиске программа должна сообщать результаты поиска, число коллизий и количество сравнений.

Вариант №13:

Разработать программу, реализующую комбинированный способ организации таблицы идентификаторов:

1. **Метод хеширования:** Метод середины квадрата.
2. **Способ разрешения коллизий:** Открытая адресация (двойное хеширование).

ХОД РАБОТЫ

В ходе лабораторной работы была разработана программа для организации хеш-таблицы с использованием метода открытой адресации.

Была реализована структура HashEntry (Рисунок 1), которая хранит значение идентификатора и состояние ячейки (state). Состояние необходимо для корректной работы алгоритма открытой адресации:

- EMPTY – ячейка пуста и никогда не использовалась;
- OCCUPIED – ячейка занята элементом;
- DELETED – элемент был удален (маркер, позволяющий не прерывать цепочку поиска).

На основе этой структуры был создан шаблонный класс HashTable, содержащий массив ячеек, текущий размер и методы для работы с таблицей.

```
// Состояния ячейки таблицы
enum CellState {
    EMPTY,      // Пусто (никогда не было данных)
    OCCUPIED,   // Занято (хранится активный элемент)
    DELETED     // Удалено (данные были, но удалены; маркер для продолжения поиска)
};

// Структура ячейки хеш-таблицы
template <typename T>
struct HashEntry {
    T value;
    CellState state;

    HashEntry() : state(EMPTY), value(T()) {}
};
```

Рисунок 1 - Структура HashEntry и состояния ячейки

На основе этой структуры был создан шаблонный класс HashTable (Рисунок 2). В приватной секции класса определены: динамический массив table (вектор), переменная capacity (размер таблицы) и size (текущее количество элементов).

```
// Шаблонный класс Хеш-таблица
template <typename T>
class HashTable {
private:
    vector<HashEntry<T>> table; // Массив ячеек
    int capacity;               // Общий размер таблицы (m)
    int size;                   // Текущее количество элементов (n)

    // Вспомогательный метод: перевод строки в числовое значение
    // Суммирует ASCII коды всех символов строки
    unsigned long long stringToLong(const string& key) const {
        unsigned long long sum = 0;
        for (char c : key) {
            sum += static_cast<unsigned char>(c);
        }
        return sum;
    }
```

Рисунок 2 - Определение класса HashTable

Для реализации задания были написаны две хеш-функции (Рисунок 3):

hash1 (основная) – реализует метод середины квадрата. Числовое представление ключа (сумма кодов символов) возводится в квадрат, после чего из результата извлекаются средние разряды.

hash2 (вспомогательная) – используется при возникновении коллизии. Она вычисляет шаг смещения для двойного хеширования.

```
int hash1(const string& key) const {
    unsigned long long num = stringToLong(key);
    unsigned long long squared = num * num;

    // Извлечение "середины". Делим на 10, чтобы отбросить последнюю цифру,
    // и берем модуль по размеру таблицы. Это простая имитация выделения средних разрядов.
    unsigned long long middle = squared / 10;

    return (int)(middle % capacity);
}

// Используется для вычисления шага смещения при коллизии.
// Должна возвращать значение > 0 и взаимно простое с capacity.
int hash2(const string& key) const {
    unsigned long long num = stringToLong(key);
    // Формула: 1 + (k mod (m-1)) гарантирует результат от 1 до m-1
    return (int)(1 + (num % (capacity - 1)));
}
```

Рисунок 3 - Методы hash1 и hash2

Были реализованы три основных метода манипуляции данными: insert (вставка), remove (удаление) и search (поиск).

1. Метод добавления элемента (Рисунок 4**)**. Метод реализует вставку с разрешением коллизий методом двойного хеширования. Алгоритм вычисляет начальный индекс $h_1(key)$ и шаг $h_2(key)$. Если ячейка занята, происходит переход к следующей ячейке по формуле: $Index_i=(h_1(key)+i\cdot h_2(key))\%Capacity$.

Важной особенностью реализации является обработка удаленных ячеек (DELETED). При вставке алгоритм запоминает первую встреченную ячейку со статусом DELETED, чтобы повторно использовать память, но продолжает поиск до встречи со статусом EMPTY, чтобы убедиться в отсутствии дубликатов ключа в таблице.

```
bool insert(const T& key) {
    if (size >= capacity) {
        cout << "Ошибка: Таблица полностью заполнена!" << endl;
        return false;
    }

    int h1 = hash1(key);
    int h2 = hash2(key);
    int index = h1;

    // Первый проход: проверяем, нет ли уже такого элемента (дубликат) и ищем место для вставки
    int firstDeletedIndex = -1; // Запоминаем первую встреченную удаленную ячейку

    for (int i = 0; i < capacity; ++i) {
        index = (h1 + i * h2) % capacity;

        if (table[index].state == OCCUPIED && table[index].value == key) {
            return false;
        }

        if (table[index].state == EMPTY) {
            if (firstDeletedIndex == -1) {
                table[index].value = key;
                table[index].state = OCCUPIED;
                size++;
                return true;
            } else {
                break;
            }
        }

        if (table[index].state == DELETED && firstDeletedIndex == -1) {
            firstDeletedIndex = index; // Запомнили место, но ищем дальше (вдруг дубликат)
        }
    }
    // Вставляем либо в найденное удаленное место, либо (если не нашли DELETED) цикл бы прервался на EMPTY.
    if (firstDeletedIndex != -1) {
        table[firstDeletedIndex].value = key;
        table[firstDeletedIndex].state = OCCUPIED;
        size++;
        return true;
    } else if (table[index].state == EMPTY) {
        // Это случай, когда цикл прервался по break (не было DELETED ранее)
        table[index].value = key;
        table[index].state = OCCUPIED;
        size++;
        return true;
    }

    return false;
}
```

Рисунок 4 - Метод добавления элемента

2. Метод удаления элемента (Рисунок 5**)**. Метод выполняет поиск элемента по алгоритму двойного хеширования. При нахождении элемента его ячейка не очищается полностью (не переводится в статус EMPTY), а помечается специальным флагом DELETED. Это необходимо для сохранения целостности цепочек поиска при открытой адресации: если сделать ячейку пустой, поиск элементов, вставленных "после" неё в результате коллизий, будет некорректно прерываться.

```
bool remove(const T& key) {
    int h1 = hash1(key);
    int h2 = hash2(key);
    int index;

    for (int i = 0; i < capacity; ++i) {
        index = (h1 + i * h2) % capacity;

        if (table[index].state == EMPTY) {
            return false; // Цепочка оборвалась, элемента нет
        }

        if (table[index].state == OCCUPIED && table[index].value == key) {
            table[index].state = DELETED; // Помечаем как удаленное
            size--;
            return true;
        }
    }
    return false;
}
```

Рисунок 5 - Метод удаления элемента

3. **Метод поиска** (Рисунок 6). Метод выполняет проход по таблице, вычисляя индексы аналогично методу вставки. Поиск продолжается до тех пор, пока не будет найден искомый элемент или пока не встретится ячейка со статусом EMPTY. Ячейки со статусом DELETED при поиске пропускаются.

Дополнительно метод ведет подсчет статистики:

- **Количество сравнений:** сколько раз программа сверила искомый ключ с содержимым ячеек.
- **Количество коллизий:** сколько раз программа обратилась к занятой ячейке, в которой находился другой элемент.

```
void search(const T& key) {
    int h1 = hash1(key);
    int h2 = hash2(key);
    int index;

    int comparisons = 0;
    int collisions = 0; // В данном контексте = число неудачных проб (i)

    bool found = false;

    cout << "\n--- Результаты поиска ---" << endl;
    cout << "Ключ: " << key << endl;
    cout << "Hash1 (нач. индекс): " << h1 << endl;
    cout << "Hash2 (шаг): " << h2 << endl;

    for (int i = 0; i < capacity; ++i) {
        index = (h1 + i * h2) % capacity;
        comparisons++; // Сравниваем состояние ячейки и значение

        if (table[index].state == EMPTY) {
            break; // Точно не найден
        }
        if (table[index].state == OCCUPIED) {
            if (table[index].value == key) {
                found = true;
                cout << "СТАТУС: НАЙДЕН по индексу " << index << endl;
                break;
            } else {
                collisions++; // Ячейка занята другим элементом
            }
        } else {
        }
    }
    if (!found) cout << "СТАТУС: НЕ НАЙДЕН" << endl;
    cout << "Количество сравнений: " << comparisons << endl;
    cout << "Количество коллизий (проходов): " << collisions << endl;
    cout << "-------------------------" << endl;
}
```

Рисунок 6 - Метод поиска с подсчетом статистики

Для взаимодействия с пользователем разработано меню (Рисунок 7-8), позволяющее выполнять все требуемые операции. Дополнительно был реализован метод printAverageStats (Рисунок 9), который рассчитывает среднее число сравнений и коллизий для всех элементов, находящихся в данный момент в таблице. Это позволяет экспериментально подтвердить теоретическую эффективность метода.

```
void showMenu() {
    cout << "\nЛАБОРАТОРНАЯ РАБОТА №5 (Вариант 13)\n";
    cout << "1. Загрузить из файла\n";
    cout << "2. Сохранить в файл\n";
    cout << "3. Поиск элемента\n";
    cout << "4. Добавить элемент\n";
    cout << "5. Удалить элемент\n";
    cout << "6. Показать таблицу\n";
    cout << "7. ОБЩАЯ СТАТИСТИКА (для отчета)\n";
    cout << "0. Выход\n";
    cout << "> ";
}
```

Рисунок 7 - Реализация меню программы (1/2)

```
while (true) {
    showMenu();
    if (!(cin >> choice)) { // Защита от ввода букв вместо цифр
        cin.clear();
        cin.ignore(10000, '\n');
        continue;
    }
    switch (choice) {
    case 1:
        cout << "Введите имя файла (напр. input.txt): ";
        cin >> buffer;
        ht.loadFromFile(buffer);
        break;
    case 2:
        cout << "Введите имя файла для сохранения: ";
        cin >> buffer;
        ht.saveToFile(buffer);
        break;
    case 3:
        cout << "Введите слово для поиска: ";
        cin >> buffer;
        ht.search(buffer);
        break;
    case 4:
        cout << "Введите слово для добавления: ";
        cin >> buffer;
        if (ht.insert(buffer)) cout << "Успешно добавлено.\n";
        else cout << "Не добавлено (дубликат или нет места).\n";
        break;
    case 5:
        cout << "Введите слово для удаления: ";
        cin >> buffer;
        if (ht.remove(buffer)) cout << "Успешно удалено.\n";
        else cout << "Элемент не найден.\n";
        break;
    case 6:
        ht.printTable();
        break;
    case 7:
        ht.printAverageStats();
        break;
    case 0:
        cout << "Выход.\n";
        return 0;
    default:
        cout << "Неверный выбор.\n";
    }
}
```

Рисунок 8 - Реализация меню программы (2/2)

```
void printAverageStats() {
    if (size == 0) {
        cout << "Таблица пуста, статистика недоступна." << endl;
        return;
    }

    long long totalComparisons = 0;
    long long totalCollisions = 0;

    for (int i = 0; i < capacity; ++i) {
        if (table[i].state == OCCUPIED) {
            // Имитируем поиск этого элемента
            T key = table[i].value;
            int h1 = hash1(key);
            int h2 = hash2(key);
            int idx;

            for (int j = 0; j < capacity; ++j) {
                idx = (h1 + j * h2) % capacity;
                totalComparisons++;

                if (table[idx].state == OCCUPIED && table[idx].value == key) {
                    // Нашли. Число коллизий для этого элемента = j
                    totalCollisions += j;
                    break;
                }
            }
        }
    }
```

Рисунок 9 - Метод printAverageStats

Ниже приведены сценарии тестирования разработанного программного обеспечения, демонстрирующие корректность работы реализованных алгоритмов хеширования и разрешения коллизий.

1. Загрузка данных и визуализация таблицы. На первом этапе тестирования была произведена загрузка списка идентификаторов из текстового файла input.txt (пункт меню 1). После успешного чтения данных содержимое хеш-таблицы было выведено на экран (пункт меню 6). Как видно на Рисунок 10, идентификаторы распределились по ячейкам массива в соответствии с индексами, вычисленными методом середины квадрата. Ячейки отображают статус OCCUPIED (занято) и хранящееся значение.

```
ЛАБОРАТОРНАЯ РАБОТА №5 (Вариант 13)
1. Загрузить из файла
2. Сохранить в файл
3. Поиск элемента
4. Добавить элемент
5. Удалить элемент
6. Показать таблицу
7. ОБЩАЯ СТАТИСТИКА (для отчета)
0. Выход
> 1
Введите имя файла (напр. input.txt): input.txt
Данные загружены из файла.
 --- Содержимое Хеш-таблицы ---
 Ind | St | Value
 ----|----|----------------
   0 | OK | private
   9 | OK | int
  13 | OK | while
  16 | OK | void
  17 | OK | return
  19 | OK | double
  20 | OK | float
  22 | OK | else
  24 | OK | class
  28 | OK | if
  29 | OK | struct
  32 | OK | for
  35 | OK | continue
  36 | OK | public
  37 | OK | break
 ------------------------------
```

Рисунок 10 - Загрузка данных из файла и вывод содержимого таблицы

2. Поиск элемента

Был выполнен поиск существующего в таблице идентификатора while через пункт меню 3. Программа вычислила хеш-функцию, определила индекс и обнаружила элемент. Результат работы представлен на Рисунок 11. Поиск был выполнен за 1 сравнение, количество коллизий составило 0, что свидетельствует об оптимальном попадании элемента в ячейку.

```
> 3
Введите слово для поиска: while

--- Результаты поиска ---
Ключ: while
Hash1 (нач. индекс): 13
Hash2 (шаг): 18
СТАТУС: НАЙДЕН по индексу 13
Количество сравнений: 1
Количество коллизий (проходов): 0
-------------------------
```

Рисунок 11 - Поиск элемента

3. Динамическое добавление данных

Для проверки операций модификации таблицы был добавлен новый уникальный идентификатор figure (пункт меню 4). После добавления был произведен контрольный поиск этого же элемента. Рисунок 12 демонстрирует, что элемент был успешно размещен в таблице (алгоритм нашел свободную ячейку) и впоследствии корректно найден функцией поиска.

```
> 4
Введите слово для добавления: figure
Успешно добавлено.
--- Содержимое Хеш-таблицы ---
Ind | St | Value
----|----|----------------
  0 | OK | private
  9 | OK | int
 11 | OK | figure
 13 | OK | while
 16 | OK | void
 17 | OK | return
 19 | OK | double
 20 | OK | float
 22 | OK | else
 24 | OK | class
 28 | OK | if
 29 | OK | struct
 32 | OK | for
 35 | OK | continue
 36 | OK | public
 37 | OK | break
------------------------------
--- Результаты поиска ---
Ключ: figure
Hash1 (нач. индекс): 11
Hash2 (шаг): 3
СТАТУС: НАЙДЕН по индексу 11
Количество сравнений: 1
Количество коллизий (проходов): 0
--------------------------
```

Рисунок 12 - Добавление и поиск нового идентификатора

4. Анализ эффективности

Для оценки производительности реализованного метода хеширования была выведена общая статистика по всей таблице (пункт меню 7). На Рисунок 13 представлены итоговые показатели: коэффициент заполнения таблицы и среднее количество сравнений. Полученное значение среднего числа сравнений (~1.5).

```
=== СТАТИСТИКА ДЛЯ ОТЧЕТА ===
Размер таблицы (m): 41
Элементов (n): 16
Коэффициент заполнения: 39.02%
Среднее число сравнений: 1.50
Среднее число коллизий:  0.50
============================
=== СТАТИСТИКА ДЛЯ ОТЧЕТА ===
Размер таблицы (m): 41
Элементов (n): 16
Коэффициент заполнения: 39.02%
Среднее число сравнений: 1.50
Среднее число коллизий:  0.50
============================
=== СТАТИСТИКА ДЛЯ ОТЧЕТА ===
Размер таблицы (m): 41
Элементов (n): 41
Коэффициент заполнения: 100.00%
Среднее число сравнений: 2.68
Среднее число коллизий:  1.68
============================
```

Рисунок 13 - Статистика эффективности хеш-таблицы

ВЫВОД

В ходе выполнения лабораторной работы была реализована программа, организующая таблицу идентификаторов с использованием метода хеширования. Были применены **метод середины квадрата** (для вычисления хеша) и **двойное хеширование** (для разрешения коллизий).
Полученные экспериментальные данные (Рисунок 13) наглядно демонстрируют зависимость производительности от коэффициента заполнения таблицы:

1. При умеренной загрузке таблицы (**39%**, 16 элементов) среднее число сравнений составило **1.50**, что приближено к идеальному показателю $O(1)$.
2. При критической загрузке таблицы (**100%**, 41 элемент), когда в таблице не осталось свободных ячеек, среднее число сравнений возросло лишь до **2.68**.

Эти результаты подтверждают высокую эффективность метода двойного хеширования.

Таким образом, разработанная хеш-таблица обеспечивает доступ к данным значительно быстрее классических структур даже в условиях полного заполнения памяти, а механизм двойного хеширования успешно предотвращает первичную кластеризацию, равномерно распределяя элементы при возникновении коллизий.